

---

# A shorter tour of R/qlt

Karl W Broman

Department of Biostatistics and Medical Informatics  
University of Wisconsin – Madison

<https://rqlt.org>

26 November 2012

---

## Preliminaries

1. To install R/qlt, type (within R) `install.packages("qlt")` (This needs to be done just once.)
2. To load the R/qlt package, type  

```
library(qlt)
```

This needs to be done every time you start R. (There is a way to have the package loaded automatically every time, but we won't discuss that here.)
3. To view the objects in your workspace:  

```
ls()
```
4. The best way to get help on the functions and data sets in R (and in R/qlt) is via the html version of the help files. One way to get access to this is to type  

```
help.start()
```

This should open a browser with the main help menu. If you then click on **Packages** → **qlt**, you can see all of the available functions and datasets in R/qlt. For example, look at the help file for the function `read.cross`.

An alternative method to view this help file is to type one of the following:  

```
help(read.cross)  
?read.cross
```

The html version of the help files are somewhat easier to read, and allow use of hotlinks between different functions.
5. All of the code in this tutorial is available as a file from which you may copy and paste into R, if you prefer that to typing. Type the following within R to get access to the file:  

```
url.show("https://rqlt.org/rqltour2.R")
```

## Data import

We will consider data from Sugiyama et al., *Physiol Genomics* 10:5–12, 2002. The data are from an intercross between BALB/cJ and CBA/CaJ; only male offspring were considered. There are four phenotypes: blood pressure, heart rate, body weight, and heart weight. We will focus on the blood pressure phenotype, will consider just the 163 individuals with genotype data and, for simplicity, will focus on the autosomes. The data are contained in the comma-delimited file <https://rqlt.org/sug.csv>.

6. Load the data into R/qlt as follows.

```
sug <- read.cross("csv", "https://rqlt.org", "sug.csv",  
                 genotypes=c("CC", "CB", "BB"), alleles=c("C", "B"))
```

The function `read.cross` is for importing data into R/qlt. `"sug.csv"` is the name of the file, which we import directly from the R/qlt website. `genotypes` indicates the codes used for the genotypes; `alleles` indicates single-character codes to be used in plots and such.

`read.cross` loads the data from the file and formats it into a special cross object, which is then assigned to `sug` via the assignment operator (`<-`).

## Diagnostics

Generally, at this point, one would spend considerable time studying the genotype and phenotype data, looking for potential errors. In many cases, about half of the analysis time is devoted to such diagnostics.

In previous tutorials, we've often gotten bogged down in this part, and so we'll skip it here, assume that the data are okay, and jump right into QTL mapping. See the longer ("brief") tour of R/qtl at <https://rqtl.org/tutorials/>, or Chapter 3 of Broman and Sen (2009).

## Summaries

The data object `sug` is complex; it contains the genotype data, phenotype data and genetic map. R has a certain amount of "object oriented" facilities, so that calls to functions like `summary` and `plot` are interpreted appropriately for the object considered.

The object `sug` has "class" `"cross"`, and so calls to `summary` and `plot` are actually sent to the functions `summary.cross` and `plot.cross`.

7. Get a quick summary of the data. (This also performs a variety of checks of the integrity of the data.)

```
summary(sug)
```

We see that this is an intercross with 163 individuals. There are 6 phenotypes, and genotype data at 93 markers across the 19 autosomes. The genotype data is quite complete.

8. There are a number of simple functions for pulling out pieces of summary information. Hopefully these are self-explanatory.

```
nind(sug)
nchr(sug)
totmar(sug)
nmar(sug)
nphe(sug)
```

9. Get a summary plot of the data.

```
plot(sug)
```

The plot in the upper-left shows the pattern of missing genotype data, with black pixels corresponding to missing genotypes. The next plot shows the genetic map of the typed markers. The following plots are histograms or bar plots for the six phenotypes. The last two "phenotypes" are sex (with 1 corresponding to males) and mouse ID.

10. Individual parts of the above plot may be obtained as follows.

```
plotMissing(sug)
plotMap(sug)
plotPheno(sug, pheno.col=1)
plotPheno(sug, pheno.col=2)
plotPheno(sug, pheno.col=3)
plotPheno(sug, pheno.col=4)
plotPheno(sug, pheno.col=5)
plotPheno(sug, pheno.col=6)
```

## Single-QTL analysis

Let's now proceed to QTL mapping via a single-QTL model.

11. We first calculate the QTL genotype probabilities, given the observed marker data, via the function `calc.genoprob`. This is done at the markers and at a grid along the chromosomes. The argument `step` is the density of the grid (in cM), and defines the density of later QTL analyses.

```
sug <- calc.genoprob(sug, step=1)
```

The output of `calc.genoprob` is the same cross object as input, with additional information (the QTL genotype probabilities) inserted. We assign this back to the original object (writing over the previous data), though it could have also been assigned to a new object.

12. To perform a single-QTL genome scan, we use the function `scanone`. By default, it performs standard interval mapping (that is, maximum likelihood via the EM algorithm). Also, by default, it considers the first phenotype in the input cross object (in this case, blood pressure).

```
out.em <- scanone(sug)
```

13. The output has “class” “`scanone`”. The `summary` function is passed to the function `summary.scanone`, and gives the maximum LOD score on each chromosome.

```
summary(out.em)
```

14. Alternatively, we can give a threshold, e.g., to only see those chromosomes with  $\text{LOD} > 3$ .

```
summary(out.em, threshold=3)
```

15. We can plot the results as follows.

```
plot(out.em)
```

16. We can do the genome scan via Haley-Knott regression by calling `scanone` with the argument `method="hk"`.

```
out.hk <- scanone(sug, method="hk")
```

17. We may plot the two sets of LOD curves together in a single call to `plot`.

```
plot(out.em, out.hk, col=c("blue", "red"))
```

18. Alternatively, we could do the following:

```
plot(out.em, col="blue")
plot(out.hk, col="red", add=TRUE)
```

19. It's perhaps more informative to plot the differences:

```
plot(out.hk - out.em, ylim=c(-0.3, 0.3), ylab="LOD (HK) - LOD (EM)")
```

20. To perform a genome scan by the multiple imputation method, one must first call `sim.geno` to perform the multiple imputations. This is similar to `calc.genoprob`, but with an additional argument, `n.draws`, indicating the number of imputations. We then call `scanone` with `method="imp"`.

```
sug <- sim.geno(sug, step=1, n.draws=64)
out.imp <- scanone(sug, method="imp")
```

21. We may plot all three curves together as follows.

```
plot(out.em, out.hk, out.imp, col=c("blue", "red", "green"))
```

22. We can plot the LOD curves for just chromosomes 7 and 15 as follows.

```
plot(out.em, out.hk, out.imp, col=c("blue", "red", "green"), chr=c(7,15))
```

23. We can also look at differences.

```
plot(out.imp - out.em, out.hk - out.em, col=c("green", "red"), ylim=c(-1,1))
```

## Permutation tests

To perform a permutation test, to get a genome-wide significance threshold or genome-scan-adjusted p-values, we use `scanone` just as before, but with an additional argument, `n.perm`, indicating the number of permutation replicates. It's quickest to use Haley-Knott regression.

24. In case the time to perform the permutation test is too long, you can skip it (here) and load the results (that I calculated previously) for this plus other time-consuming stuff we'll see shortly as follows.

```
load(url("https://rqtl.org/various.RData"))
```

25. The code to do the actual permutation test is the following:

```
operm <- scanone(sug, method="hk", n.perm=1000)
```

26. A histogram of the results (the 1000 genome-wide maximum LOD scores) is obtained as follows:

```
plot(operm)
```

27. Significance thresholds may be obtained via the `summary` function:

```
summary(operm)
summary(operm, alpha=c(0.05, 0.2))
```

28. Most importantly, the permutation results may be used along with the `scanone` results to have significance thresholds and p-values calculated automatically:

```
summary(out.hk, perms=operm, alpha=0.2, pvalues=TRUE)
```

### Interval estimates of QTL location

For the blood pressure phenotype, we've seen good evidence for QTL on chromosomes 7 and 15. Interval estimates of the location of QTL are commonly obtained via 1.5-LOD support intervals, which may be calculated via the function `lodint`. Alternatively, an approximate Bayes credible interval may be obtained with `bayesint`.

29. To obtain the 1.5-LOD support interval and 95% Bayes interval for the QTL on chromosome 7, type:

```
lodint(out.hk, chr=7)
bayesint(out.hk, chr=7)
```

The first and last rows define the ends of the intervals; the middle row is the estimated QTL location.

30. It is sometimes useful to identify the closest flanking markers; use `expandtomarkers=TRUE`:

```
lodint(out.hk, chr=7, expandtomarkers=TRUE)
bayesint(out.hk, chr=7, expandtomarkers=TRUE)
```

31. We can calculate the 2-LOD support interval and the 99% Bayes interval as follows.

```
lodint(out.hk, chr=7, drop=2)
bayesint(out.hk, chr=7, prob=0.99)
```

32. The intervals for the chr 15 locus may be calculated as follows.

```
lodint(out.hk, chr=15)
bayesint(out.hk, chr=15)
```

### QTL effects

We may obtain plots indicating the estimated effects of the QTL via `plotPXG`, which creates a dot plot, or `effectplot`, which plots the average phenotype for each genotype group.

33. For `plotPXG`, we must first identify the marker closest to the QTL peak. Use `find.marker`.

```
max(out.hk)
mar <- find.marker(sug, chr=7, pos=47.7)
plotPXG(sug, marker=mar)
```

Note that red dots correspond to inferred genotypes (based on a single imputation).

34. The function `effectplot` uses the multiple imputation results from `sim.geno`.

```
effectplot(sug, mname1=mar)
```

35. We may use `effectplot` at a position on the "grid" between markers, using "7@47.7" to indicate the position at 47.7 cM on chr 7.

```
effectplot(sug, mname1="7@47.7")
```

36. Similar plots may be obtained for the locus on chr 15.

```
max(out.hk, chr=15)
mar2 <- find.marker(sug, chr=15, pos=12)
plotPXG(sug, marker=mar2)
effectplot(sug, mname1="15@12")
```

37. We may plot the joint effects of the two loci via `plotPXG` as follows:

```
plotPXG(sug, marker=c(mar, mar2))
plotPXG(sug, marker=c(mar2, mar))
```

38. The function `effectplot` gives more readable figures in this case; it's often useful to look at it in both ways.

```
effectplot(sug, mname1="7@47.7", mname2="15@12")
effectplot(sug, mname2="7@47.7", mname1="15@12")
```

The two loci do not appear to interact.

## Other phenotypes

By default in `scanone`, we consider the first phenotype in the input cross object. Other phenotypes, include the parallel consideration of multiple phenotypes, can be considered via the argument `pheno.col`.

39. To analyze the second phenotype, refer to it by its numeric index, as follows.

```
out.hr <- scanone(sug, pheno.col=2, method="hk")
```

40. Alternatively, refer to a phenotype by its name:

```
out.bw <- scanone(sug, pheno.col="bw", method="hk")
```

41. You can also give a numeric vector of phenotype values. This is useful for considering a transformed version of a phenotype, such as log body weight.

```
out.logbw <- scanone(sug, pheno.col=log(sug$pheno$bw), method="hk")
```

42. Use of vector of phenotype indices results in an object with multiple LOD score columns, one for each phenotype.

```
out.all <- scanone(sug, pheno.col=1:4, method="hk")
```

43. For this final case, it's important to note that the `summary` function, by default, focuses solely on the first LOD score column.

```
summary(out.all, threshold=3)
```

Here, it looks at the first LOD score column and picks off the peaks that are above 3, and then gives the LOD scores at that location for the other three columns.

To do the same thing but focusing on another column, use the argument `lodcolumn`.

```
summary(out.all, threshold=3, lodcolumn=4)
```

44. Alternatively, use `format="allpeaks"`, to get the maximum LOD score for each column, with a chromosome being shown if at least one of the LOD score column exceeds the threshold.

```
summary(out.all, threshold=3, format="allpeaks")
```

45. A third version of the output is obtained with `format="allpheno"`, which gives one row per LOD peak and gives the LOD scores for all columns at each peak.

```
summary(out.all, threshold=3, format="allpheno")
```

46. There are two other formats that might be preferred: `format="tabByCol"` and `format="tabByChr"`. These give tables with one significant LOD peak per phenotype, organized either by phenotype (with `"tabByCol"`) or by chromosome (with `"tabByChr"`). The tables include 1.5-LOD support intervals, and so one may wish to use `"tabByCol"` even if there is only one LOD score column.

```
summary(out.all, threshold=3, format="tabByCol")
```

```
summary(out.all, threshold=3, format="tabByChr")
```

## Two-dimensional, two-QTL scans

Two-dimensional, two-QTL scans offer the opportunity to detect interacting loci or to separate pairs of linked QTL. Analysis is performed with `scantwo`, which is much like `scanone`.

47. For 2d scans, it's advantageous to run things at a coarser step size, by first re-running `calc.genoprob`.

```
sug <- calc.genoprob(sug, step=2)
```

48. To perform a 2d scan for the blood pressure phenotype, use the following. If you loaded the file `"various.RData"` in step 24 on page 3, you can skip this, as you already have the results.

```
out2 <- scantwo(sug, method="hk")
```

49. We may plot the results as follows.

```
plot(out2)
```

The upper-triangle contains interaction LOD scores, comparing the full two-locus model to the additive two-locus model. The lower-triangle contains the “full” LOD scores, comparing the full two-locus model to the null model. Because of the clear evidence for QTL on chromosomes 7 and 15, we see “tails” along those two chromosomes: the two locus model with either chr 7 or chr 15 and anything else is clearly better than the null model.

50. It's best to replace the lower-triangle with the LOD score comparing the full model to the best single-QTL model, using either `lower="cond-int"` or `lower="fv1"` (the two are equivalent).

```
plot(out2, lower="fv1")
```

51. We can also look at the LOD scores comparing the additive two-QTL model to the best single-QTL model, using either `upper="cond-add"` or `upper="av1"`.

```
plot(out2, lower="fv1", upper="av1")
```

52. To assess significance, we need to do a permutation test. This can be extremely time consuming. The results were already loaded in step 24 on page 3, but here is the code (though here I cite `n.perm=5` rather than `n.perm=1000`, as I'd recommend).

```
operm2 <- scantwo(sug, method="hk", n.perm=5)
```

53. With the permutation results in hand, we can get a summary with p-values.

```
summary(out2, perms=operm2, alpha=0.2, pvalues=TRUE)
```

The pair of loci on 7 and 15 are clear. They show no evidence for an interaction. There is some evidence for an additional locus on chr 12, with  $p=0.17$ .

## Multiple-QTL analyses

After performing the single- and two-QTL genome scans, it's best to bring the identified loci together into a joint model, which we then refine from which we may explore the possibility of further QTL. In this effort, we work with “QTL objects” created by `makeqtl`. We fit multiple-QTL models with `fitqtl`. A number of additional functions will be introduced below.

53. Let's re-run `calc.genoprob` so that we are working at a step size of 1 cM again.

```
sug <- calc.genoprob(sug, step=1)
```

54. First, we create a QTL object containing the loci on chr 7 and 15.

```
qtl <- makeqtl(sug, chr=c(7,15), pos=c(47.7, 12), what="prob")
```

The last argument, `what="prob"`, indicates to pull out the QTL genotype probabilities for use in Haley-Knott regression.

55. We fit the two locus additive model as follows.

```
out.fq <- fitqtl(sug, qtl=qtl, method="hk")
summary(out.fq)
```

A key part of the output is the “drop one term at a time” table, which compares the fit of the two-QTL model to the reduced model in which a single QTL is omitted.

56. We may obtain the estimated effects of the QTL via `get.ests=TRUE`. We use `dropone=FALSE` to suppress the drop-one-term analysis.

```
summary(fitqtl(sug, qtl=qtl, method="hk", get.ests=TRUE, dropone=FALSE))
```

Since this is an intercross, we obtain estimates of the additive effect and dominance deviation for each locus.

57. To assess the possibility of an interaction between the two QTL, we may fit the model with the interaction, indicated via a model “formula”. The QTL are referred to as `Q1` and `Q2` in the formula, and we may indicate the interaction in a couple of different ways.

```
out.fqi <- fitqtl(sug, qtl=qtl, method="hk", formula=y~Q1*Q2)
out.fqi <- fitqtl(sug, qtl=qtl, method="hk", formula=y~Q1+Q2+Q1:Q2)
summary(out.fqi)
```

We don’t have time to cover the use of such formulas in any detail here. Note that there is no evidence for an interaction.

58. Another way to assess interactions is with the function `addint`, which adds one interaction at a time, in the context of a multiple-QTL model. This is most useful when there are more than two QTL being considered.

```
addint(sug, qtl=qtl, method="hk")
```

59. The locations of the two QTL are as estimated via the single-QTL scan. We may refine our estimates of QTL location in the context of the multiple-QTL model via `refineqtl`. This function uses a greedy algorithm to iteratively refines the locations of the QTL, one at a time, at each step seeking to improve the overall fit.

```
rqtl <- refineqtl(sug, qtl=qtl, method="hk")
rqtl
```

The location of each QTL changed slightly, and the overall LOD score increased by 0.03.

60. We can re-run `fitqtl` to get the revised drop-one-term table.

```
summary(out.fqr <- fitqtl(sug, qtl=rqtl, method="hk"))
```

61. The `plotLodProfile` function plots LOD profiles obtained during the call to `refineqtl`. These give one-dimensional views of the precision of QTL localization, in the context of the multiple-QTL model.

```
plotLodProfile(rqtl)
```

For each position on the curve for the chr 7 QTL, we compare the two-QTL model with the chr 7 locus in varying position but with the chr 15 locus fixed at its estimated position, to the single-QTL model with just the chr 15 locus. The chr 15 curve is similar.

These are actually slightly lower than the curves obtained from the single-QTL analysis with `scanone`.

```
plot(out.hk, chr=c(7,15), col="red", add=TRUE)
```

62. The function `addqtl` is used to scan for an additional QTL to be added to the model.

```
out.aq <- addqtl(sug, qtl=rqtl, method="hk")
```

The biggest peaks are on chr 8 and 12, but nothing is particularly exciting.

```
plot(out.aq)
```

There is also a function `addpair`, for scanning for a pair of QTL to be added.

63. Finally, we consider the function `stepwiseqtl`, which is our fully automated stepwise algorithm to optimize the penalized LOD scores of Manichaikul et al. (2009). We first need to calculate the appropriate penalties from the two-dimensional permutation results.

```
print(pen <- calc.penalties(operm2))
```

We then run `stepwiseqtl`, using `max.qtl=5`. It will perform forward selection to a model with 5 QTL, followed by backward elimination, and will report the model giving the largest penalized LOD score. The output is a QTL object.

```
out.sq <- stepwiseqtl(sug, max.qtl=5, penalties=pen, method="hk", verbose=2)
out.sq
```

With `verbose=2`, we get an indication of the location of the QTL at each step.

The result is exactly the model that we had after `refineqtl`.

## References

- Sugiyama F, Churchill GA, Li R, Libby LJM, Carver T, Yagami K-I, John SWM, Paigen B (2002) QTL associated with blood pressure, heart rate, and heart weight in CBA/CaJ and BALB/cJ mice. *Physiol Genomics* 10:5–12
- Broman KW, Sen Ś (2009) A guide to QTL mapping with R/qlt. Springer
- Manichaikul A, Moon JY, Sen Ś, Yandell BS, Broman KW (2009) A model selection approach for the identification of quantitative trait loci in experimental crosses, allowing epistasis. *Genetics* 181:1077–1086
- Dalgaard P (2008) Introductory statistics with R, 2nd edition. Springer
- Venables WN, Ripley BD (2002) Modern applied statistics with S, 4th edition. Springer